
Age of Empires II DE on iPad

A better route through the blockers

Research brief for the AgePad project owner | 8 September 2026

Recommendation: reopen targeted binary compatibility work. Keep the supplied original ARM64 engine, adapt its operating-system boundaries, and use genuine mobile middleware where possible. The evidence does not justify concluding that engine source is mandatory. It also does not yet establish that a complete retail-binary port will work.

The desired result remains the actual game running locally on iPad, with touch, sustained performance and a real supported multiplayer peer. Streaming, a substitute classic engine, a successful loader and a lobby screenshot are not equivalent outcomes.

Three findings that change the next action

1. The signing diagnosis was too strong. Apple's published implementation explains why a Mac-style process-identity lookup can fail on a non-Mac platform regardless of certificate. Waiting for a development certificate is not the best next Simulator experiment. See page 2.

2. A real mobile networking component is available. An inspection performed during this research found that Microsoft's official PlayFab Party 1.10.25 iOS package exports all 157 Party function names requested by this game, in both ARM64 device and Simulator slices. This is symbol coverage, not a proven drop-in replacement. See page 3.

3. Our startup adapter is incomplete in a consequential way. It calls the original launch delegate directly but does not broadcast AppKit's normal launch notifications. The original startup trace recorded no Steam-library load before the crash. We have not proven that the separate SDK failure explains that exact crash. See page 2.

What is actually proven

The prior probes executed original ARM64 startup in Simulator and verified selected original GPU computations. They did not produce a game frame, a gameplay FPS measurement or an iPad multiplayer session. This research adds source analysis and a static vendor-library audit; it does not add a successful game run.

Scope and assumptions

The starting input is the supplied Steam Mac build 1.1.2. No engine source, unreleased build, publisher credentials or alternative-store purchase is assumed. Existing private inputs and installed candidates were preserved during research. Recommendations distinguish immediate local experiments from routes needing external access.

Evidence: existing AgePad de-001 through de-033 records; current AppLifecycleCompat.m; new party-export-audit.json. Public sources and limitations follow beside each finding.

1. Correct the diagnosis before changing the architecture

The code-identity error has a platform explanation

Apple documents that `SecCodeCopySelf` returns a representation of the calling code whether or not it is signed. In Apple's published Security source, that function calls `autoLocateGuest`. The useful body of `autoLocateGuest` is guarded by `TARGET_OS_OSX`; the non-macOS path ends in `errSecCSNoSuchCode`. That matches the -67065 observed inside Simulator. The source was pinned to commit `db15acbe6a7f257a859ad9a3bb86097bfe0679d9`. [Apple API documentation](#); [Apple Security implementation](#).

Interpretation: unsupported platform behavior is a stronger explanation than a missing development certificate. The published source has not been matched instruction-for-instruction to this installed Simulator framework, so this is a strongly supported explanation rather than an exact binary proof. The separate static-signature trust error remains a separate observation.

A faithful adapter must represent the actual process and honor actual validation requirements. It cannot substitute a static file object for a live process, return unconditional success, or claim that a re-signed app still has Feral's original signing identity. First determine which callers need identity and which perform validation.

We may be stopping at our own launch-sequence defect

AppKit posts `will-finish` and `did-finish` notifications, and automatically registers applicable delegate methods as observers. Our adapter sets a weak delegate and invokes `didFinish` directly; that does not notify other observers or reproduce normal initialization. [Apple NSApplication documentation](#); [launch notification contract](#).

The local original-engine dyld trace contains no `Steam-library` load, while the isolated SDK probe explicitly loads that library and fails initialization. These are different execution paths. A null `Steam-related` context can result from skipped initialization as well as failed initialization. Missing launch notifications are a testable hypothesis, not a demonstrated fix.

Next experiment: compare real native startup with the adapted startup: application creation, nib wiring, notification delivery, entry into the original delayed launch routine, dynamic-library loads and actual `Steam` initialization. Correct one observed adapter divergence at a time. Count notifications and delegate delivery to avoid duplicate initialization.

Sources: Apple, current API documentation, accessed 8 Sep 2026; Apple OSS Security commit dated 16 Oct 2025. Local evidence: `de-020`, `de-023`, `de-025/026`, `de-030/032` and `AppLifecycleCompat.m`. No game runtime changed in this research.

2. Replace eligible middleware with genuine iOS builds

The Mac executable is already ARM64. The most economical hypothesis is to preserve its game code and replace only incompatible platform components. Original-binary hosting is a real technique: LiveContainer documents signed, JIT-less loading of adapted iOS executables. Its guests already use iOS APIs, so it proves a loading mechanism, not Mac-game compatibility. [LiveContainer architecture](#).

New static audit: PlayFab Party

Microsoft publishes distinct macOS and iOS frameworks. Its release history records ARM64 iOS Simulator support and XCFramework packaging. The current examined release is 1.10.25, published 28 July 2026. [Official releases](#); [Apple-platform setup](#).

Comparison performed locally	Result
Original game imports beginning with Party	157 C symbols
Official iOS ARM64 library exports matching those imports	157 of 157
Official Simulator ARM64 library exports matching those imports	157 of 157
Missing requested symbol names in either slice	0
Version string found in supplied Mac Party binary	1.10.22; not queried at runtime

The downloaded XCFramework archive is 12,201,717 bytes, SHA-256 49efca1dc45a88c3046a3d7353c247ce26a626ac9a7367eef970bffe3cbab5b0. It was downloaded from Microsoft's official GitHub release. Inspection used nm and otool; the package was not loaded into the game.

Why this matters: we may not need to port this vendor's desktop audio/network environment at all. The engine requests a C interface, making a precise compatibility audit more tractable than reconstructing an opaque C++ object hierarchy.

What remains: compare structure layouts, enum values, calling conventions, callback ownership, version behavior and all dynamically requested APIs. Matching symbol names cannot establish these. Then test genuine loading and a harmless documented query before changing the original-engine candidate. Resolve the 1.10.22-to-1.10.25 difference explicitly.

This does not fix Steam. Party multiplayer also needs the title's real configuration and legitimate player entity credentials. A sample under a new private title would prove that sample only. [Microsoft Party Quickstart](#).

Apply the same audit principle to the other vendor components, including Xbox identity and HTTP libraries, without assuming that similarly named iOS binaries are available or interchangeable.

Sources: Microsoft/PlayFab release assets and current documentation, accessed 8 Sep 2026; LiveContainer current README. Local audit: party-engine-imports.json and party-export-audit.json. No ABI, rendering or multiplayer pass is claimed.

3. Separate the Steam problem from local game execution

Idea A: a narrow Mac service helper for research

Keep simulation and rendering in the iPad-target process, but let a separate Mac helper call the genuine Steam SDK against the logged-in desktop client. Return actual values and callbacks over a narrowly scoped local connection. This is a proposed experiment, not an existing supported Steam deployment.

The first test should be genuine initialization plus one harmless read-only result, with real failure propagation. Only then consider the exact interface subset the original engine uses. Process-local pointers and C++ objects cannot simply be passed between processes; callback ordering and object lifetimes require explicit handling.

Purpose: determine whether desktop integration can be separated from the original engine's local execution. No video or simulation would be streamed. Nevertheless, this branch would still require a Mac and cannot pass the standalone iPad goal. It must not expand into a permanent dependency by accident.

Valve requires successful initialization before using Steam interfaces; the desktop client provides their implementations. Our native control satisfies that dependency, whereas the Simulator probe does not. [Valve Steamworks API overview](#).

Idea B: investigate the genuine alternate-store build

The official AoE announcement says a Mac App Store release is planned later in 2026. No current listing or supplied MAS build was verified in this research. A legitimate non-Steam distribution is a high-value future input because it may avoid the desktop Steam dependency. It will still need its real receipt, identity and platform behavior. Changing the Steam build's distribution flags is not equivalent. [Official Mac announcement](#).

Feral already operates Calico for supported mobile multiplayer. However, its general FAQ says Mac App Store Calico multiplayer does not interoperate with Steam Mac games. We do not know whether AoE's future MAS edition will use Calico or what its peer cohort will be. **A Steam-free build could improve startup while narrowing multiplayer compatibility.** [Feral Calico FAQ](#).

Idea C: ask for a smaller enabling component

Publisher cooperation need not mean obtaining the entire engine source. An authorized build with mobile-capable platform libraries, a supported identity adapter, or an appropriate test build could remove a particular boundary. These are proposed requests; nobody was contacted.

An open-source packet library alone is insufficient. Valve's GameNetworkingSockets separates transport from Steam authentication and other services. Its existence does not authorize this retail title's lobbies. [Valve networking project](#).

Sources: Valve and Feral current documentation; AoE support update 30 Apr 2026; accessed 8 Sep 2026. The helper and alternative-build routes remain untested hypotheses.

4. Ideas considered beyond the current approach

Static recompilation: real precedent, weak immediate benefit

M-HT/SR translates original DOS/Windows game executables with game-specific analysis and runtime code, including targets for macOS ARM64. It is a counterexample to the claim that original source is always required. It is not a generic DE converter. For an already-ARM64 engine, lifting every instruction adds semantic risk without removing desktop service calls. Reserve it for a demonstrated instruction or ABI obstacle. [SR project](#).

Windows DE through Wine and x86 translation

This could become relevant if Windows multiplayer is essential, but it adds CPU translation, Windows API support and graphics translation. BoxiOS reports a physical-iPhone Linux Powder Toy demonstration at about 4.5-5 FPS with JIT disabled. That is the maintainer's measurement, not ours, and proves neither Wine nor Steam nor DE. It is not evidence that a fast Windows DE stack is available for iPad. [BoxiOS checkpoint](#).

Run an entire Mac inside the iPad

The located VirtualMacOniPad project targets specific older iPads on iPadOS 14-16.3.1. UTM's ordinary sideloaded iOS build lists no hypervisor, and its documented QEMU GPU acceleration is not evidence of accelerated macOS Metal gaming. These projects do not supply a current stock-iPad solution for this game. [VirtualMacOniPad](#); [UTM iOS support](#); [UTM display support](#).

Adopt an existing Mac compatibility layer wholesale

Darling targets Linux and documents limited GUI compatibility. Our prior macOS audit found missing imported classes and behavior that reports success for unimplemented UI. These are sources of implementation ideas, not established game runtimes. [Darling limitations](#); [maciOS](#).

Apple's Game Porting Toolkit

The current Toolkit 4 includes porting examples and Metal tools. Its Windows evaluation environment and source-oriented examples do not automatically convert a retail Mac app into an iPad app. Use the tools for targeted renderer work and measurement rather than treating the toolkit as a missing emulator. [Apple Game Porting Toolkit](#).

Ranking: targeted ARM64 adaptation first; genuine mobile middleware and alternate distribution next; Mac helper only for isolation; full static lifting or Windows emulation as conditional fallbacks; no current justification for full-Mac virtualization.

Sources: maintainer documentation and Apple, accessed 8 Sep 2026. No third-party execution demo was reproduced. No unverified project advertisement is treated as a working solution.

5. Touch, performance and multiplayer acceptance

Simulator is a workflow target, not the hardware verdict

Apple explains that Simulator uses a separate userspace and Mach bootstrap namespace on the Mac kernel. Its GPU path can expose fewer or different capabilities than a physical Apple GPU. Keep the requested Simulator gameplay milestone, but also test device-target loading and representative shaders early when a provisioned iPad is available. A Simulator failure can be artificial; a Simulator success can conceal device restrictions. [Apple Simulator architecture](#); [Apple Metal Simulator guidance](#).

Measure real gameplay frame times, not diagnostic refresh. Retain the target of 60 FPS and initial 30 FPS floor at a recorded resolution and scenario; report median/p95 frame times, sustained duration, memory and thermal behavior. Separate host, Simulator and device measurements. No numerical FPS forecast for DE on iPad is justified yet.

The approximately 20 GB supplied content inventory is disk data, not a measured RAM requirement. Most game data has not been staged in the Simulator candidate. Resource access, streaming and actual engine memory use remain work, even if startup services are solved.

Touch should feed the original game's input

Proposed starting controls: tap selection, a distinct contextual-order gesture, two-finger camera movement, pinch mapped to supported game zoom, an explicit drag-selection mode, and a visible queue-command modifier. Construction, training and control groups need reachable touch targets using the original visual style. These are design proposals requiring real gameplay testing, not implemented controls.

Keep commands flowing through the original engine so the touch layer does not replace game rules or networking. Test cancelled gestures, multi-touch conflicts, tooltips, small targets, save/resume and interruption behavior. Multiplayer must not depend on a local tactical-pause convenience.

Feral's Company of Heroes demonstrates that a full RTS can have touch-oriented command-panel/wheel controls and mobile multiplayer. Its implementation is precedent, not evidence that its code can be transplanted into AoE. [Feral mobile RTS FAQ](#).

Multiplayer is more than getting packets through

The official native Mac DE release supports Mac multiplayer and excludes other-platform cross-play. An iPad adaptation must still prove that a supported retail peer accepts it and that a sustained game stays synchronized. Matching CPU architecture or library symbols cannot prove admission or simulation equivalence. Preserve this distinction if a MAS-based route changes the peer population. [Official multiplayer scope](#).

Sources: Apple and Feral current documentation; AoE support update 30 Apr 2026; accessed 8 Sep 2026. Control proposals and acceptance measurements are our engineering recommendations.

6. The next bounded investigation loop

Start with experiments that distinguish explanations. Do not repeat unchanged SDK failures or add unrelated UI while startup remains unproven. The following are recommended next actions, not completed implementation.

A. Reconstruct the real launch contract

Compare native and Simulator startup, including nib wiring, launch notifications, run-loop order and the first real Steam initialization attempt. Add result-preserving observations; keep secrets out of logs. Pass: identify and correct the first demonstrated adapter divergence and observe its effect in original code. If startup still fails identically, retain the evidence and move to B rather than call it a launch pass.

B. Audit code-identity semantics

Identify what the original caller requests and which validation requirements follow. Use Apple's published implementation to design a faithful current-process representation if the required operations can be supported. Pass: results agree with the real native control for the same process/property, including failures. An identity lookup success alone does not pass signature validation, ownership or startup.

C. Qualify the official iOS Party library

Extend the completed 157-symbol coverage check to ABI layouts, callback ownership and version behavior. Load the official platform slice in an isolated candidate and perform harmless documented queries. Only then substitute it in a private original-engine build. Pass: real operations work without adapter stubs; a multiplayer pass still requires the actual game's authorized session.

D. Isolate Steam only if the trace reaches it

If the original engine demonstrably needs a desktop-only service, prototype the smallest real-result Mac helper. Pass: actual initialization and the specific required read-only operation, then callback ordering. Label every result Mac-dependent. Do not report the branch as standalone iPad success or expand into a general Steam reimplementing without evidence of value.

E. Add a device check and evaluate alternate inputs

When hardware/signing are available, test ordinary signed ARM64 loading and representative original shaders on the iPad, before investing in Simulator-only graphics fixes. Separately inspect a genuine MAS/developer build when available, including its real receipts and supported peers. These experiments have external prerequisites; A-C do not require the full engine source.

Final acceptance remains unchanged

Actual menus and an AI match in the designated Simulator; measured gameplay performance; touch-alone play on a physical iPad; real saves, audio and lifecycle; a sustained match with a supported retail peer without desynchronization. None is replaced by source research, symbol matches or a host-assisted demonstration.

Research limits and stopping point

The source-backed diagnoses and current public package audit now identify different, actionable tests. Additional broad searches were yielding weaker examples rather than a ready DE port. Remaining uncertainty is chiefly experimental: launch causality, full ABI compatibility, real title authentication and device execution. The report therefore stops at a ranked test program, not a claim of feasibility or a delivery-date estimate.

Source notes

Primary sources accessed 8 September 2026. Links open the source page. Living documentation has no publication date asserted unless stated below.

Apple - [SecCodeCopySelf\(_:_:\)](#)

Undated living documentation; publication date not asserted. Used on report page(s): 2.

Apple - [Security / Code.cpp, autoLocateGuest implementation](#)

2025-10-16. Used on report page(s): 2.

Apple - [NSApplication](#)

Undated living documentation; publication date not asserted. Used on report page(s): 2.

Apple - [didFinishLaunchingNotification](#)

Undated living documentation; publication date not asserted. Used on report page(s): 2.

Project maintainers - [LiveContainer README - How does it work?](#)

Undated living documentation; publication date not asserted. Used on report page(s): 3.

Microsoft / PlayFab - [PlayFab Party releases](#)

2026-07-28 (latest examined release). Used on report page(s): 3.

Microsoft / PlayFab - [iOS and macOS getting started - PlayFab](#)

Undated living documentation; publication date not asserted. Used on report page(s): 3.

Microsoft / PlayFab - [Party Quickstart](#)

Undated living documentation; publication date not asserted. Used on report page(s): 3.

Valve - [Steamworks API Overview](#)

Undated living documentation; publication date not asserted. Used on report page(s): 4.

Age of Empires Support - [Age of Empires II: Definitive Edition coming to Mac](#)

2026-04-30 (updated). Used on report page(s): 4, 6.

Feral Interactive - [Calico FAQs](#)

Undated living documentation; publication date not asserted. Used on report page(s): 4.

Valve - [GameNetworkingSockets README](#)

Undated living documentation; publication date not asserted. Used on report page(s): 4.

Source notes, continued

Primary sources accessed 8 September 2026. Links open the source page. Living documentation has no publication date asserted unless stated below.

Project maintainers - [SR - static recompilation of DOS and Windows games](#)

Undated living documentation; publication date not asserted. Used on report page(s): 5.

Project maintainers - [BoxiOS README - release 0.1.0 build126](#)

2026-04-27 (reported checkpoint). Used on report page(s): 5.

Project maintainers - [VirtualMacOniPad README](#)

Undated living documentation; publication date not asserted. Used on report page(s): 5.

Project maintainers - [iOS - UTM installation and support matrix](#)

Undated living documentation; publication date not asserted. Used on report page(s): 5.

Project maintainers - [Display - UTM QEMU settings](#)

Undated living documentation; publication date not asserted. Used on report page(s): 5.

Project maintainers - [Known non-functional software - Darling Docs](#)

Undated living documentation; publication date not asserted. Used on report page(s): 5.

Project maintainers - [maciOS README](#)

Undated living documentation; publication date not asserted. Used on report page(s): 5.

Apple - [Game Porting Toolkit 4](#)

Undated living documentation; publication date not asserted. Used on report page(s): 5.

Apple - [Getting the Most Out of Simulator - WWDC19](#)

WWDC 2019. Used on report page(s): 6.

Apple - [Developing Metal apps that run in Simulator](#)

Undated living documentation; publication date not asserted. Used on report page(s): 6.

Feral Interactive - [Company of Heroes for mobile - FAQs](#)

Undated living documentation; publication date not asserted. Used on report page(s): 6.